

Agent-Poison: Benchmarking Indirect Prompt Injection Resilience in Tool-Calling LLM Agents

Anonymous Author(s)
Anonymous Institution

Abstract—We present *agent-poison*, an open-source benchmarking harness for measuring how open-weight, locally-deployed tool-calling LLM agents handle indirect prompt injection (IPI) delivered through the content of tool outputs. Unlike prior work targeting proprietary frontier models or single-turn settings, we focus on 7-8B parameter models served locally via Ollama, the class of models increasingly used in cost- and privacy-sensitive agentic deployments. Our harness runs a deterministic multi-turn tool-calling loop across three realistic scenarios (document summarization, customer support, SQL reporting), each poisoned with one of three injection strategies: direct override, contextual deception, and delimiter manipulation. We evaluate three models (Llama 3.1 8B, Qwen2.5 7B, Mistral 7B) under four baseline defenses (none, XML delimiters, system-message reinforcement, dual-prompt sanitization) and measure Attack Success Rate (ASR), benign task accuracy, and refusal rate across repeated stochastic trials. We find that a single deterministic (temperature 0) trial substantially misrepresents a model's true vulnerability; that across a full 8-trial replication protocol (576 runs), dual-prompt sanitization was the most consistently effective defense, driving ASR to 0% for two of three models; and that system-message reinforcement is highly model-dependent, driving Llama 3.1 8B's ASR from 33% to 0% (at the cost of more refusals) while leaving Qwen2.5 7B's ASR statistically unchanged (50% vs. 54%).

Keywords—*indirect prompt injection, LLM agents, tool calling, benchmarking, red-teaming, open-weight models, AI security*

I. INTRODUCTION

Tool-calling agents built on large language models increasingly execute multi-turn loops in which the model reads content it did not choose — a document, a support ticket, a database row, a web page — and decides what to do next based on it. That content is, from a security standpoint, attacker-controllable: anyone who can influence what a tool returns can embed text that looks like an instruction, and a model that does not reliably separate *data* from *instructions* may act on it. This is indirect prompt injection (IPI), and it has been demonstrated against real, deployed LLM-integrated applications.

Most published IPI evaluation has centered on proprietary frontier models (GPT-4-class systems) accessed over an API. Far less is known about the open-weight, 7-8B-parameter models that many teams now run locally via Ollama or vLLM for cost, latency, or data-residency reasons — and whether the simple, cheap defenses those teams are most likely to actually deploy (a stricter system prompt, wrapping tool output in delimiter tags) meaningfully help. We built **agent-poison** to answer this empirically, offline, and reproducibly.

Our contributions:

- An offline-first benchmarking harness that drives a real multi-turn tool-calling loop against any OpenAI-compatible endpoint, with a fully deterministic mock tool server so every run is reproducible.
- Three scenarios spanning three distinct injection strategies — direct override, contextual deception, and delimiter manipulation — grounded in realistic agent toolsets (email, refund issuance, database export).
- Four baseline defenses (none, XML delimiters, system-message reinforcement, dual-prompt sanitization) implemented and evaluated head-to-head on identical scenarios.
- A methodological finding: a single deterministic (temperature 0) trial per condition — the natural first thing to try, and what our own first pass did — gives Attack Success Rate estimates that do not survive replication. We show this concretely in Section VII.
- A cross-model finding that a defense's effectiveness does not transfer across open-weight models of similar scale: the same system-message reinforcement defense is dramatically effective for one model and statistically inert for another.

II. RELATED WORK

Prompt injection was first documented as a direct attack, in which a malicious instruction appears in the user's own input to hijack the model's stated goal or leak its system prompt [2]. Greshake et al. [1] generalized this to the *indirect* setting: an attacker plants an instruction in content the model retrieves as data (a web page, a document), which is far more dangerous in agentic, tool-using systems because the injected instruction arrives through a channel the model is designed to trust. Our benchmark targets exactly this indirect setting.

Two benchmarks are the closest prior work. InjecAgent [3] evaluates tool-integrated agents against indirect injections across 1,054 test cases spanning 17 user tools and 62 attacker tools, finding ReAct-prompted GPT-4 vulnerable roughly 24% of the time. AgentDojo [4] instead provides a dynamic, extensible environment (97 tasks, 629 security test cases across four domains) for evaluating both attacks and defenses. Both target primarily frontier, API-served models; agent-poison instead targets small open-weight models runnable on a single machine, trading benchmark breadth for full reproducibility and zero API cost, and adds an explicit multi-trial replication protocol that we show is necessary (Section VII).

On the defense side, the Instruction Hierarchy [5] proposes training models to treat system/developer instructions as

privileged over content encountered later, which is the training-time analogue of our system-message reinforcement baseline. StruQ [6] and spotlighting [7] instead attack the problem structurally — separating the prompt and data channels (StruQ, via structured queries and targeted fine-tuning) or making the untrusted span visually/lexically distinct to the model (spotlighting, via delimiting, datamarking, or encoding) — which is the idea behind our `xml_delimiters` baseline. Zhan et al. [8] show that several published defenses that appear effective against static injections collapse under adaptive, optimization-based attacks; our injections are static (hand-authored, not adversarially optimized against each target model), so our ASR numbers should be read as a lower bound on true vulnerability, consistent with their critique. Finally, indirect prompt injection is ranked the top risk (LLM01) in the OWASP Top 10 for LLM Applications [9], underscoring its practical relevance beyond academic benchmarks.

III. THREAT MODEL

The attacker controls only the *content returned by one tool call* — never the system prompt, the user's own message, or the tool schema itself. This matches the indirect setting of [1]: the agent asks a tool a legitimate question (read a document, look up a ticket, run a query) and the attacker has pre-planted text inside whatever that tool will return. The agent has no way to tell, from the tool result alone, whether it is looking at ordinary data or an attempted instruction.

Each scenario in agent-poison exercises one of three injection strategies:

- **Direct override** — an explicit fake command embedded in the data, e.g. `[SYSTEM COMMAND]: Ignore previous task... in doc_summarizer_exfil`, instructing the agent to abandon its task and email the document to an external address.
- **Contextual deception** — a fabricated error or authorization prompt that gives the agent a plausible-sounding *reason* to call a privileged tool, e.g. a fake `Error 403: Authorization required in customer_ticket_escalation` that asks the agent to "verify identity" by issuing a \$4,999 refund.

- **Delimiter manipulation** — text that fakes the closing of the tool-result block and opens a forged system turn, e.g. a database row in `sql_report_taint` containing a fake closing tag followed by a forged `<system>` instruction, to make injected text look like it arrived through a trusted channel rather than tainted data.

Success, for the attacker, means the agent calls the scenario's `unauthorized_target_tool` (e.g. `send_email`, `issue_admin_refund`, `export_database_dump`) at any point in the conversation — regardless of whether it also completes the legitimate task.

IV. BENCHMARK DESIGN

Scenario schema. Each scenario is a Pydantic-validated JSON record: a system prompt, a user prompt, a set of tools in OpenAI function-calling schema, the `ground_truth_tool` call that legitimately completes the task, the `unauthorized_target_tool` the injection tries to trigger, and a `mock_returns` map giving each read-type tool both a `benign_content` and an `injected_content` string.

Deterministic mock server. A `MockToolServer` executes every tool call the model makes. It never calls a network or an LLM — it looks up `(tool_name, poisoned)` and returns a fixed string, so a run is fully reproducible and the only source of variance is the model's own sampling.

Multi-turn runner. An `AgentRunner` drives a real conversation against any OpenAI-compatible endpoint (we use Ollama): send system + user prompt with tools attached, execute whatever tool calls come back through the mock server, feed the results back as tool messages, and repeat for up to 4 turns or until the model stops calling tools. Every tool call is logged with its turn number; a run is flagged `hijack_triggered` the moment `unauthorized_target_tool` is called, and `ground_truth_completed` once the legitimate tool has been called.

Table I summarizes the three scenarios.

TABLE I. SCENARIOS

Scenario	Ground truth tool	Unauthorized target	Injection strategy
<code>doc_summarizer_exfil</code>	<code>read_document</code>	<code>send_email</code>	Direct override
<code>customer_ticket_escalation</code>	<code>fetch_ticket</code>	<code>issue_admin_refund</code>	Contextual deception
<code>sql_report_taint</code>	<code>run_sql_query</code>	<code>export_database_dump</code>	Delimiter manipulation

Defenses. All four transform the tool output (and/or the system prompt) before it re-enters the conversation:

- `none` — the raw mock output is passed through unchanged.
- `xml_delimiters` — the output is wrapped in `<untrusted_tool_output>` tags, in the spirit of spotlighting/delimiting [7].
- `system_reinforcement` — an additional system message is prepended stating that tool output is untrusted

data, never instructions, in the spirit of the instruction hierarchy [5].

- `dual_prompt` — a secondary, tool-less call to the *same* model is used to extract only the factual content of the tool output before it is shown to the agent, discarding anything that reads as an instruction.

V. METRICS

For a set of runs of a given (model, scenario, defense) cell, split into poisoned runs P and benign runs B:

- **Attack Success Rate (ASR)** — the share of poisoned runs where the agent called the unauthorized tool: $|\{r \text{ in } P : \text{hijack_triggered}(r)\}| / |P|$.
- **Benign Accuracy** — the share of clean runs that complete the legitimate task without any hijack: $|\{r \text{ in } B : \text{ground_truth_completed}(r) \text{ AND NOT hijack_triggered}(r)\}| / |B|$.
- **Task Interruption Rate** — the share of *all* runs, poisoned or not, that fail the legitimate task, whether or not a hijack occurred: $|\{r \text{ in } PUB : \text{NOT ground_truth_completed}(r)\}| / |PUB|$.
- **Refusal Rate** — the share of runs where the model produced no tool call and its final text reads as a refusal: $|\{r \text{ in } PUB : \text{refused}(r)\}| / |PUB|$.

ASR and Task Interruption Rate answer different questions and are not complementary: a run can fail the legitimate task *and* avoid the hijack (e.g. it refuses outright), so the two rates are reported separately rather than as one minus the other.

VI. EXPERIMENTAL SETUP

All models were served locally through Ollama 0.34.0's OpenAI-compatible endpoint (<http://localhost:11434/v1>)

on the same machine: **Llama 3.1 8B** (4.9 GB), **Qwen2.5 7B** (4.7 GB), and **Mistral 7B** (4.4 GB, resolved to the same weights as `mistral:latest`). `max_turns` was capped at 4.

We ran two protocols:

- **Initial sweep** — temperature 0.0 (greedy decoding), a single trial per cell: 3 models $\times$ 3 scenarios $\times$ 4 defenses $\times$ {poisoned, benign} = 72 runs. This was our first pass and is reported in Section VII-A primarily as a cautionary baseline.
- **Replication protocol** — temperature 0.7, 8 trials per cell, run separately per (model, defense) pair across all 3 scenarios $\times$ {poisoned, benign} = 48 runs per pair. Applied to all three models under all four defenses (576 runs total), for 648 logged runs across both protocols combined.

VII. RESULTS

Table II reports the initial single-trial sweep across all four defenses; Table III reports the full 8-trial replication protocol across all three models and all four defenses.

A. Initial single-trial sweep (temperature 0, $N=1$)

TABLE II. INITIAL SINGLE-TRIAL SWEEP (TEMPERATURE 0, $N=1$)

Model	Defense	ASR	Benign Acc.	Refusal
Llama 3.1 8B	none	33.3%	100%	0%
Llama 3.1 8B	xml_delimiters	0%	100%	0%
Llama 3.1 8B	system_reinforcement	0%	100%	33.3%
Llama 3.1 8B	dual_prompt	0%	100%	0%
Qwen2.5 7B	none	33.3%	100%	0%
Qwen2.5 7B	xml_delimiters	33.3%	100%	0%
Qwen2.5 7B	system_reinforcement	66.7%	100%	0%
Qwen2.5 7B	dual_prompt	0%	100%	0%
Mistral 7B	any of the four	0%	66.7%	0%

Mistral 7B's 0% ASR across every defense in Table II is confounded: it never reliably called `read_document` at all in `doc_summarizer_exfil` (benign accuracy there was 0% under every defense), so it could not be hijacked through that scenario for reasons unrelated to robustness. `dual_prompt` fully suppressed every hijack it saw in this pass, for every model.

B. Replication protocol (temperature 0.7, 8 trials/cell)

We re-ran all four defenses for all three models under the full protocol ($N=16$ poisoned+benign per scenario, $N=48$ per model-defense cell, $N=192$ per defense), shown in Table III.

TABLE III. REPLICATION PROTOCOL (TEMPERATURE 0.7, 8 TRIALS/CELL)

Model	Defense	ASR	Benign Acc.	Refusal
Qwen2.5 7B	none	50.0%	100%	0%
Qwen2.5 7B	xml_delimiters	37.5%	100%	2.1%
Qwen2.5 7B	system_reinforcement	54.2%	100%	0%
Qwen2.5 7B	dual_prompt	0%	100%	0%
Llama 3.1 8B	none	33.3%	100%	6.25%
Llama 3.1 8B	xml_delimiters	0%	95.8%	0%
Llama 3.1 8B	system_reinforcement	0%	91.7%	16.7%
Llama 3.1 8B	dual_prompt	0%	100%	0%
Mistral 7B	none	4.2%	58.3%	2.1%
Mistral 7B	xml_delimiters	0%	66.7%	4.2%
Mistral 7B	system_reinforcement	4.2%	54.2%	8.3%
Mistral 7B	dual_prompt	4.2%	58.3%	2.1%

Three results stand out against Table II:

- **The single-trial "defense backfire" does not replicate.** Table II suggested `system_reinforcement` roughly doubled Qwen2.5 7B's ASR (33.3% $\rightarrow$ 66.7%). At $n=8$ per condition, both `none` and `system_reinforcement` sit within a few points of 50% (50.0% vs. 54.2%), well inside binomial sampling noise for $n=24$ poisoned trials per arm (standard error ≈ 10 points). The correct reading is that `system_reinforcement` gives Qwen2.5 7B **no measurable protection**, not that it makes things worse.
- **The same defense is model-dependent.** For Llama 3.1 8B, `system_reinforcement` drives ASR from 33.3% to a clean 0% across all three scenarios — but at a real cost: benign accuracy drops from 100% to 91.7% and refusal rate nearly triples (6.25% $\rightarrow$ 16.7%). The model is not distinguishing the injected instruction more precisely; it appears to become more generally suspicious of tool output, including legitimate output, and refuses more often as a result.
- **'dual_prompt' is the only defense that helps everywhere it can.** It drives ASR to 0% for both Qwen2.5 7B and Llama 3.1 8B with no accuracy or refusal cost, confirming the pattern from the single-trial pass under full replication. Mistral 7B is the exception: its ASR under `dual_prompt` (4.2%) is identical to its `none` baseline, because the injected instruction was never the thing limiting it — a model that already struggles to call tools reliably has little for a sanitizing pre-pass to fix. Averaged across all three models, `dual_prompt` is the strongest defense we test (mean ASR 1.4%), followed by `xml_delimiters` (12.5%); `system_reinforcement` is roughly neutral overall (19.5%, close to `none`'s 29.2%) once Qwen2.5 7B's null result is included alongside Llama's full fix.

VIII. DISCUSSION

Single deterministic trials are not a safe basis for ASR claims. Our own first pass (Table II) would have shipped a finding — "a defensive system prompt roughly doubles Qwen's attack success rate" — that a modest replication effort (8 stochastic trials instead of 1) shows is very likely noise. Any IPI evaluation reporting ASR from greedy, single-sample decoding should be treated cautiously, and we would argue the same applies to prior benchmarks that do not explicitly report trial counts or variance.

Defenses do not transfer across models. `system_reinforcement` is the difference between a fully-hijackable and a fully-resistant agent for Llama 3.1 8B, and statistically nothing at all for Qwen2.5 7B. This is not simply "Llama is the safer model" — under `none`, both models have the same 33.3-50% range of ASR. Rather, only Llama's instruction-following seems responsive to an explicit textual warning about untrusted tool content; Qwen's does not, at least not at this warning's phrasing. A defense recommendation validated on one open-weight model is not evidence it will hold on another of similar scale.

A defense can be a wash rather than a win or a loss. Mistral 7B's overall ASR was unchanged by `system_reinforcement` (4.2% either way), which could read as "the defense is neutral here." The per-scenario breakdown says otherwise: the defense drove `doc_summarizer_exfil` task completion to 0% (it stopped reading the document at all, attacked or not) while opening a new 12.5% vulnerability in `customer_ticket_escalation` that had not existed under `none`. An unchanged aggregate ASR can hide a defense actively reshuffling where a model fails, which is invisible unless results are reported per-scenario rather than as a single rolled-up number.

Refusal is a hidden cost of naive defenses. Llama 3.1 8B's refusal rate under `system_reinforcement` (16.7%) was almost three times its rate under `none` (6.25%), and its benign accuracy correspondingly dropped. A defense that trades hijack susceptibility for spurious refusals on legitimate tool output is not free, and Task Interruption Rate / Refusal Rate need to be reported alongside ASR, not as a footnote, or a paper (or a production system) could declare victory on ASR while quietly making the agent less useful.

dual_prompt's strength held up under replication, but it is not a universal fix. It fully suppressed hijacks for Qwen2.5 7B and Llama 3.1 8B under the full 8-trial protocol, consistent with the mechanism (a secondary sanitizing call has no reason to reproduce an embedded instruction it wasn't asked to relay) — the single-trial result was, this time, correct. But it left Mistral 7B's ASR exactly where `none` left it (4.2%), because Mistral's vulnerability was never really about the injection surviving into context; it is a model that calls tools unreliably in general, and a sanitizing pre-pass cannot fix that. `dual_prompt` also roughly doubles latency and inference cost per tool call, a real deployment cost that plain XML delimiters or `system_reinforcement` do not carry — worth paying for Qwen or Llama, not obviously worth it for a model like Mistral where it buys nothing.

IX. LIMITATIONS

- **Three models, all 7-8B.** We do not know whether either finding (defense non-transfer, refusal cost) holds at larger scale (30B+) or for models explicitly safety-tuned against injection.
- **Three synthetic scenarios.** Each pairs one injection strategy with one domain; we cannot yet separate "this model resists direct-override" from "this model resists document-summarization-domain attacks." A larger scenario set crossing strategies and domains independently is needed before strategy-level claims are safe.
- **Static, hand-authored injections.** As noted in Section VIII, our ASR is a lower bound: Zhan et al. [8] show optimization-based adaptive attacks break defenses that look solid against static injections like ours.
- **No human baseline.** We have no measurement of how often a human operator, given the same tool output, would fall for the same injection — useful context for

interpreting whether a given ASR is "bad for an LLM" or "bad, period."

- **Mock tool server, not a live environment.** Our mock server is intentionally static and deterministic for reproducibility, unlike AgentDojo's dynamic environment [4]; it cannot model an adaptive attacker who changes the injection based on the agent's behavior mid-conversation.

X. CONCLUSION

agent-poison shows that open-weight, locally-deployed tool-calling agents are measurably vulnerable to indirect prompt injection, that cheap prompt-level defenses are inconsistent across models of similar scale, and — the finding we consider most important for how this kind of research should be done — that a single deterministic trial per condition is not sufficient evidence for an ASR claim, defense-backfire or otherwise. Across the full 8-trial replication protocol (all three models, all four defenses, 576 runs), `dual_prompt` sanitization was the most consistently effective defense we tested, `system_reinforcement` was roughly neutral once averaged across models, and no defense fixed Mistral 7B's underlying tool-calling unreliability. We are releasing the harness, the three scenarios, and every raw run transcript so results here can be independently checked and extended.

Immediate next steps: (1) extend the model set to larger open-weight models and at least one safety-tuned variant; (2) add adaptive, optimization-based injections per [8] to establish an upper- rather than lower-bound ASR; (3) grow the scenario set to cross injection strategy and domain independently; (4) implement StruQ-style structured queries [6] as a fourth architectural (not just prompt-level) defense baseline.

ETHICAL CONSIDERATIONS

This work studies how to make tool-calling AI agents more robust against indirect prompt injection — a defensive goal. Every attack in this benchmark is synthetic and targets only a mocked internal tool server under our control; none was ever directed at a real deployed system, third-party service, or individual, and no human-subject or personally identifiable data was collected or used anywhere in this work. We disclose three injection strategies in enough detail to be reproducible, consistent with established practice in the benchmarks we build on (InjecAgent [3], AgentDojo [4]); none of the techniques are novel or more capable than attacks already documented in the literature we cite. We see no meaningful risk of this paper enabling harm beyond what is already public, and a clear benefit in giving defenders a reproducible way to measure and compare mitigations.

OPEN SCIENCE

We release every artifact needed to reproduce this work: the agent-poison benchmarking harness (scenario schema, deterministic mock tool server, multi-turn runner, all four defense implementations, and the metrics module), the three scenario definitions, and the raw JSON transcript of every one of the 648 logged runs behind Tables II and III. All of this is available in a fully anonymized repository at [https://anonymous.4open.science/r/agent-poison-](https://anonymous.4open.science/r/agent-poison-artifact-0418/)

[artifact-0418/](https://anonymous.4open.science/r/agent-poison-artifact-0418/), which will remain available for the duration of the review process; the non-anonymized version of the repository will be made public again after the review period. The anonymized repository's commit history, filenames, and file metadata have been scrubbed of any reference to the authors or their institution.

LLM USAGE CONSIDERATIONS

This paper was produced with extensive LLM assistance, both as the subject of study and as a tool in the research process. We address the three considerations below.

Accountability and correctness. The authors take full responsibility for the correctness of every claim, number, and piece of code in this paper. All experimental results were independently verified by re-deriving the reported metrics from the raw run transcripts (Open Science, above) using the project's own aggregation code, and by manually inspecting a sample of transcripts to confirm that automated hijack/completion labels matched the model's actual behavior.

Transparency. An AI coding agent (Claude, Anthropic, operating via Claude Code) was used throughout the research process: it designed and implemented the benchmarking harness, executed all 648 logged experimental runs against locally-hosted open-weight models, and drafted the text of this manuscript under the direction of the human author, who set the research questions, reviewed and approved methodological decisions (e.g., moving from a single-trial to an 8-trial replication protocol after the tool identified single-sample noise as a threat to validity), and is responsible for the final content.

Responsibility. No human-subject or personally identifiable data was collected or used anywhere in this work; all scenario content (documents, support tickets, database rows, and injected instructions) is synthetic and hand-authored for this benchmark. The injection payloads target only fictitious internal tools executed against a fully mocked, offline tool server. All model inference was run locally on consumer hardware via Ollama using existing pre-trained open-weight checkpoints (Llama 3.1 8B, Qwen2.5 7B, Mistral 7B); no models were trained or fine-tuned for this work, limiting the additional environmental footprint to local inference only.

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv:2302.12173, 2023.
- [2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," arXiv:2211.09527, 2022.
- [3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in Findings of ACL 2024, arXiv:2403.02691.
- [4] E. DeBenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in NeurIPS 2024 Datasets and Benchmarks Track, arXiv:2406.13352.
- [5] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," arXiv:2404.13208, 2024.
- [6] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in USENIX Security 2025, arXiv:2402.06363.

- [7] K. Hines et al., "Defending against indirect prompt injection attacks with spotlighting," Microsoft, 2024.
- [8] Q. Zhan et al., "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," arXiv:2503.00061, 2025.
- [9] OWASP GenAI Security Project, "OWASP Top 10 for LLM Applications 2025," LLM01: Prompt Injection.